程序-全栈测试题

面试题目：广告ROI数据分析系统

app_roi_data.csv

项目背景

这是一个基于真实移动应用广告投放数据的面试题目。用于考察候选人的Web开发能力和数据可视化。

数据说明

数据文件

- `app_roi_data.csv` - 主要数据文件

数据结构

列名	说明	示例
日期	投放日期	2025-04-13(日)
app	应用名称	App-1, App-2, App-3, App-4, App-5
出价类型	广告出价类型	CPI
国家地区	投放国家	美国, 英国
应用安装.总次数	当日应用安装次数	140
当日ROI	当日投资回报率	6.79%
1日ROI	1日累计ROI	14.24%
3日ROI	3日累计ROI	27.3%
7日ROI	7日累计ROI	64.44%
14日ROI	14日累计ROI	120.89%
30日ROI	30日累计ROI	214.65%
60日ROI	60日累计ROI	368.42%
90日ROI	90日累计ROI	413.81%

数据特点

- **时间范围**: 2025年4月13日 - 2025年7月12日（3个月）

- **总记录数**: 910条

- **应用数量**: 5个（App-1到App-5）

- **投放国家**: 美国、英国

- **出价类型**: 统一为CPI（每次安装成本）

重要数据说明

- **0%数据的两种含义**:

1. **真实0%**: 该日期的实际ROI确实为0%（无收益或亏损）
2. **日期不足0%**: 该日期到数据截止日期的时间间隔不足对应的ROI统计周期

- **日期不足0%的具体说明**:

- 如果某日期距离数据截止日期不足90天，则90日ROI显示为0%
- 如果某日期距离数据截止日期不足60天，则60日ROI显示为0%
- 如果某日期距离数据截止日期不足30天，则30日ROI显示为0%
- 以此类推...

- **数据处理建议**:

- 需要区分这两种0%的情况，可以在数据库中添加标识字段
- 在图表中对"日期不足0%"可以选择隐藏数据点，或者用不同样式标记
- 在悬停提示中说明0%的具体原因（真实0%或日期不足）

数据存储要求

- **重要**: 提供的CSV数据必须导入到数据库系统中进行存储和查询

- **不允许**: 直接读取CSV文件进行数据查询和展示

- **数据库设计**: 需要设计合理的数据表结构来存储ROI数据

- **数据导入**: 实现CSV数据批量导入数据库的功能

页面设计规范

整体布局

- **页面标题**: "App-X - 多时间维度ROI趋势"（X为选中的应用编号）

- **副标题**: "(7日移动平均)"

- **数据范围**: "数据范围: 最近90天"

筛选控制区域

- **第一行筛选器**（从左到右）：

- 用户安装渠道（下拉选择：Apple）
- 出价类型（下拉选择：CPI）
- 国家地区（下拉选择：美国、英国等）
- APP（下拉选择：App-1、App-2、App-3、App-4、App-5）

- **第二行控制器**（两组单选框，水平排列）：

- 数据显示模式：◯ 显示移动平均值 ◯ 显示原始数据
- Y轴刻度：◯ 线性刻度 ◯ 对数刻度

图表区域

- **图表类型**: 多条趋势线图

- **数据线**（8条）：

- 当日(7日均值)
- 1日(7日均值)
- 3日(7日均值)
- 7日(7日均值)
- 14日(7日均值)
- 30日(7日均值)
- 60日(7日均值)
- 90日(7日均值)

- **预测数据**: 虚线样式（相同颜色）

- **100%回本线**: 红色水平线，标注"100%回本线"

- **X轴**: 时间标签（日期格式）

- **Y轴**: 百分比标签

- **图例**: 显示所有数据线的颜色和名称

交互功能

- 下拉选择器切换时，图表数据和标题相应更新

- 单选框切换时，图表显示模式相应变化
- 鼠标悬停显示具体数值
- 图例点击可切换数据线显示/隐藏

题目要求

基础要求（必须完成）

1. **数据库设计和数据导入**

- 设计合理的数据表结构来存储ROI数据
- 考虑如何标识和区分真实0%和日期不足0%的数据
- 实现CSV文件批量导入数据库功能

2. **数据读取与展示**

- 通过数据库查询获取ROI数据（不允许直接读取CSV文件）
- 实现基本的数据筛选功能（按应用、国家、日期范围）
- 设计合理的API接口提供数据查询服务

3. **ROI趋势图表**

- 绘制多时间维度的ROI趋势图（当日、1日、3日、7日、14日、30日、60日、90日）
- 支持按应用和国家筛选
- 图表需要清晰易读，有图例说明
- 添加100%回本线（红色水平线，标注"100%回本线"）
- 支持预测数据显示（虚线样式）
- 图表需要包含X轴时间标签和Y轴百分比标签
- 正确处理0%数据（区分真实0%和日期不足0%，并给出不同的处理方案）

4. **页面布局和交互控制**

- 实现如图所示的页面布局和样式
- 顶部筛选器：用户安装渠道、出价类型、国家地区、APP（从左到右排列）
- 数据显示模式和Y轴刻度的单选框控制（水平排列）

5. **详细设计和使用文档**（基础要求）

  - **系统设计文档**：

    - 整体架构设计说明

    - 数据库表结构设计

  - API接口设计规范
  - 前端组件架构说明
- **使用说明文档：**
  - 项目安装和运行指南
  - 数据导入操作指南
  - 图表功能使用指南
  - 筛选器和控制器操作说明
- **部署文档：**
  - 环境配置要求（数据库配置）
  - 数据库初始化步骤
  - 项目配置说明
  - 生产环境部署流程

进阶要求（加分项）

1. **高级图表功能**
   - 支持对数/线性坐标切换（如图所示的单选框控制）
   - 支持移动平均线显示（如图所示的单选框控制）
   - 支持数据点悬停显示详情
   - 图例点击切换显示/隐藏对应数据线
2. **数据分析功能**
   - ROI预测功能（基于历史数据）
   - 移动平均计算
   - 趋势分析和异常检测
   - 智能处理0%数据点（区分真实0%和日期不足0%，分别处理）
3. **用户体验优化**
   - 响应式设计
   - 暗色模式支持
4. **技术架构**
   - 前后端分离架构
   - API设计规范

- 代码结构清晰，有注释
- 错误处理完善

技术栈建议

基于本项目的技术架构，推荐使用以下技术栈：

推荐方案

- **前端**: Next.js 14 + React 18 + TypeScript + Tailwind CSS + ShadCN/UI + Recharts
- **后端**: Node.js + Express.js + TypeScript（必需，用于数据API）
- **数据库**: MySQL 8.0+ 或 PostgreSQL 13+
- **数据处理**: 后端处理CSV导入，前端调用API获取数据
- **渲染方式**: CSR（客户端渲染）

前端技术栈

- **核心框架**: Next.js 14 + React 18 + TypeScript
- **样式方案**: Tailwind CSS（原子化CSS，快速开发）
- **UI组件库**: ShadCN/UI（现代化组件库，基于Radix UI）
- **图表库**: Recharts（React生态系统中的优秀图表库）
- **状态管理**: React Hooks + Context API
- **开发工具**: ESLint + Prettier

后端技术栈（必需）

- **运行环境**: Node.js 18+
- **Web框架**: Express.js + TypeScript
- **CSV处理**: csv-parser, fast-csv 等
- **API设计**: RESTful API

数据库技术栈

- **主数据库**: MySQL 8.0+ 或 PostgreSQL 13+
- **连接池**: 数据库连接池管理
- **数据库设计**: 设计ROI数据表结构
- **数据导入**: 批量CSV数据导入功能

数据处理方案

- **数据导入**: 后端实现CSV文件解析和批量导入数据库
- **API服务**: 提供数据查询API接口，支持筛选和分页
- **前端调用**: 前端通过API获取数据并进行图表渲染

替代技术栈（如不熟悉推荐方案）

其他框架选择

- **前端框架**: Vue.js 3 + TypeScript, Angular 15+
- **后端框架**: Python (Flask/Django), Java (Spring Boot), Go (Gin)
- **数据库**: PostgreSQL, MySQL, SQLite（开发环境）
- **ORM工具**: Python (SQLAlchemy), Java (JPA/Hibernate), Go (GORM)
- **图表库**: D3.js（高度定制），ECharts（功能丰富）

开发环境要求

- Node.js 18+（推荐方案）
- MySQL 8.0+ 或 PostgreSQL 13+
- npm 或 yarn 包管理器

评分标准

功能实现 (40%)

- **数据展示功能 (20%)**：数据正确读取和展示
- **图表功能 (15%)**：图表功能完整性和交互性

- **界面交互 (5%)**：筛选器和控制器的交互功能

文档质量 (25%)

- **系统设计文档 (10%)**：架构设计、数据结构设计的完整性和清晰度
- **使用说明文档 (8%)**：用户指南的详细程度和可操作性
- **部署文档 (7%)**：部署步骤的完整性和准确性

代码质量 (20%)

- 代码结构和组织
- 命名规范和注释
- 错误处理和边界情况
- 代码规范和最佳实践

用户体验 (10%)

- 界面美观度
- 交互流畅性
- 响应式设计
- 操作流程的用户友好性

技术深度 (5%)

- 技术选型合理性
- 性能优化
- 扩展性考虑
- 数据处理的高效性

提交要求

1. **代码仓库**: 提供完整的项目代码（github地址）
2. **演示**: 部署地址
3. **文档包**（基础要求）：
   - **README.md**: 项目启动方式和基本说明

- **DESIGN.md**: 系统设计文档
- **USER_GUIDE.md**: 用户使用指南
- **DEPLOYMENT.md**: 部署说明文档

4. **AI提示词导出**（如果使用了AI辅助开发）：
5. **演示视频**（可选）: 展示完整的功能和图表交互

文档模板要求

DESIGN.md 应包含：

- 系统整体架构图
- 数据库表结构设计和ER图
- API接口设计规范
- 前端组件层次结构
- 数据流向图（CSV导入→数据库→API→前端）

USER_GUIDE.md 应包含：

- 项目功能介绍
- 数据导入操作指南
- 图表操作指南
- 筛选功能使用方法
- 控制器使用说明
- 界面截图说明

DEPLOYMENT.md 应包含：

- 环境依赖清单（Node.js、数据库等）
- 数据库安装和配置
- 项目安装步骤
- 数据库初始化和CSV数据导入
- 项目启动方式
- 生产环境配置

预估完成时间

- **数据库设计和数据导入**: 1-2小时
- **API接口开发**: 1-2小时
- **前端数据展示**: 1-2小时
- **图表功能实现**: 2-3小时
- **界面布局和交互**: 1-2小时
- **文档编写**: 1-2小时
- **进阶功能**: 1-2小时
- **优化完善**: 1小时

总计：9-16小时（可分阶段完成）

注意事项

1. 必须使用数据库存储数据，不允许直接读取CSV文件
2. 重点考察全栈开发能力，包括数据库设计、API开发和前端可视化
3. 图表功能是核心要求，必须实现多条ROI趋势线和100%回本线
4. 数据导入功能是基础要求，必须实现CSV批量导入
5. 代码质量和用户体验同样重要
6. 可以使用任何开源库和框架
7. 鼓励创新和个人风格的体现
8. 详细的文档是基础要求，不可忽视

联系方式

如有技术问题或需求澄清，请联系面试官。

祝您面试顺利！